RIPS-Frontier 🌳

RIPS-frontier (Reachability via Incremental Pruning of Subtrees) is a dynamic graph algorithm that maintains a spanning tree of the reachable set under structural churn (edge insertions and deletions).
This repository includes the core Python implementation, an automated test suite, and an adaptive monitoring controller that switches between frontier maintenance and batched BFS based on workload ratios.

🚀 Features

- **Exact Single-Source Reachability:** Maintains an exact reachability tree under continuous edge insertions and deletions.
- **Bounded Update Work:** Every effective update is bounded by V + 2m work units (where vertex resets and edge examinations count as work units).
- **Adaptive Strategy Controller:** Automatically monitors action-to-query ratios (r = A/Q) and switches between local frontier maintenance and batched BFS to optimize cumulative cost.
- **Fully Tested:** Includes automated randomized test suites verifying correctness against a BFS oracle.

📂 Project Structure

- rips_frontier.py — Core RipsFrontier class implementing insertions, deletions, subtree pruning, and adaptive switching.
- test_rips.py — Automated validation test suite executing randomized runs and checking invariants.
- main.py — Example demonstration script running operations and queries.
- LICENSE — Open-source MIT license.
- .gitignore — Ignore rules for Python cache and environment files.

💻 Quick Start

1. Basic Usage

```
from rips_frontier import RipsFrontier

# Initialize the graph with source 's' and vertex capacity
rf = RipsFrontier(s='s', n_vertices=10)

# Insert edges
rf.insert('s', 'a')
rf.insert('a', 'b')

# Query reachability
```

```
print(rf.query('b'))  # Returns: True
print(rf.query('c'))  # Returns: False

# Delete an edge (triggers subtree repair if a tree arc is hit)
rf.delete('s', 'a')
```

2. Running Demonstration & Tests

Run the sample script:
```
python main.py
```

Run the validation suite:
```
python test_rips.py
```

📝 License

This project is open-source under the MIT License.